

Full-Stack Engineer Take-Home

Plan for about 60 minutes. Use the AI tools you actually code with (Cursor, Claude Code, Copilot, v0). We assume the AI writes most of the code, so that isn't what we're reading for. We're reading for the judgment on top of it: where you caught it being wrong, what you did when the spec was fuzzy, and whether you checked your own work.

You are not expected to finish everything in the time. Prioritize. We evaluate your judgment, the depth of whatever you choose to go deep on, and your writeups, not raw completion. A focused, well-reasoned partial submission beats a rushed attempt at all of it.

Send a short `AI_LOG.md` with the code: the tools you used, the few prompts that mattered, and a couple of places the AI got it wrong that you had to fix. We read the submission on its own, with no call, so let your writeups (this one, plus `DECISIONS.md`, `REVIEW.md`, and `THREATS.md` below) carry the weight.

What to build

A chatbot, deployed on Vercel (or anywhere with a live URL), plus a public repo. The background color of each reply bubble depends on what the user typed. Check these in order, first match wins:

1. A city and a temperature in Celsius. Color between deep blue and bright red by temperature: deep blue at 0 or below, light purple around 15, bright red at 35 or above.
2. Otherwise, a standalone decimal number. Grayscale or sepia ramp from the first two decimal digits: .00 lightest, .99 darkest.
3. Otherwise, ask the LLM how urgent or panicked the message sounds, and color it from violet for high panic, through magenta, to pale yellow for completely calm.

Hard requirements:

- A real LLM on the backend, not canned replies. You can shut the project off a week after you send it.
- It works with a keyboard, and the text stays readable as the background color shifts. We care about this one, and we'll open the app and check it ourselves. Watch keyboard focus as new bubbles arrive: when a reply appends to the thread, a keyboard user should not lose their place or get yanked around.
- Sign-in, and only `@petasight.com` emails get in. The backend endpoint must enforce this rule, not just the login page.

Stretch (you may not reach this, and that is fine): each person sees only their own chat history. We store messages per user. A signed-in person must not be able to read or change another person's messages through the API, even with a valid `@petasight.com` session of their own. Treat the user's identity as

something your server establishes, not something the client hands you. This is the deepest part of the test; if you only get partway, say where you got to and how you would close the gap in `THREATS.md`.

The part we actually read for

- One rule above is underspecified on purpose. A message can land on more than one rule at the same time. Picture something like "Austin 21.5 we need to leave right now" typed in a hurry: it reads as a city with a temperature, it carries a standalone decimal, and it sounds panicked, all three at once. The order says first match wins, but think about whether order alone is the right call here, or whether one of these signals should win for a reason. Don't email us to ask. Pick an answer, build it, and say why in a `DECISIONS.md`. Name the case you chose and walk us through the call.
- Write a `THREATS.md`. Spell out how one signed-in user could try to reach another user's messages through your API, what you did (or would do) to stop it, and how you convinced yourself it holds. If you treated the per-user isolation above as a stretch and didn't fully build it, write the threat model anyway: name the attack and the defense you'd ship. Short is fine. We want the thinking, not a checklist.
- The `review/` folder has a small module and its test with a few bugs. Tell us what's broken, why, and how you'd fix it, in a `REVIEW.md`.

That's the whole thing. We're reading for your judgment, how you handle the ambiguous bit, how you reason about who can touch whose data, how you read someone else's code, and whether you tested your own work. Not how many features you packed in.

(Bonus, only if you're enjoying it: have the bot reply in a right-to-left language, in the voice of a historical philosopher or scientist from that culture, original script then an English translation.)